

Dynamic Programming (I)

- Fibonacci Numbers
- Minimum Coin Change

Yanlin Zhang & Shangqi Lu | DSAA 2043 Fall 2026

- Definition

$$f(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F(n-1) + F(n-2) & \text{if } n > 1 \end{cases}$$

- The first several numbers are:
 - 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89 , 144 ...
- Question: Given n , how to compute $F(n)$?
 - Recursion



Leonardo
Fibonacci

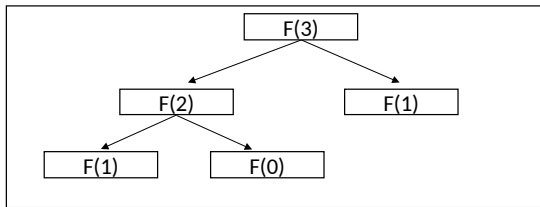
Fibonacci Numbers – Naïve Algorithm

- Computing the n^{th} Fibonacci number recursively:

$$f(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ F(n-1) + F(n-2) & \text{if } n > 1 \end{cases}$$

code ...

```
def Fib(n):  
    if (n <= 1)  
        return n;  
    else  
        return Fib(n - 1) + Fib(n - 2);
```



- Running time

$$T(n) = T(n-1) + T(n-2) + O(1)$$

$$\Rightarrow T(n) \geq T(n-1) + T(n-2) \text{ for } n \geq 2$$

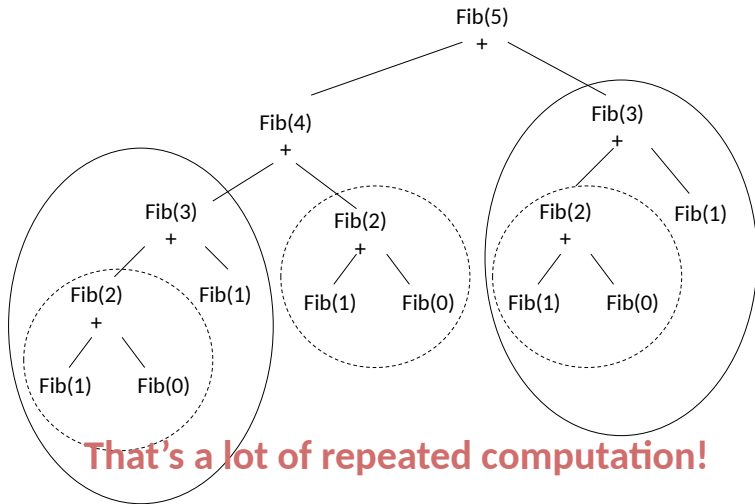
$$\Rightarrow T(n) \geq 2T(n-2)$$

- What is the solution to this?

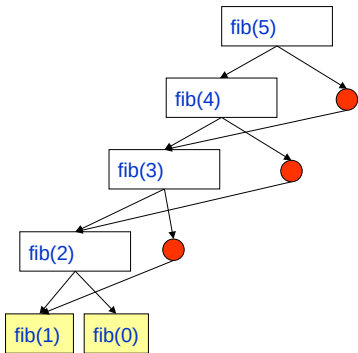
- Clearly it is $O(2^n)$, but this is not tight.
- A lower bound is $\Omega(2^{n/2})$.
- You should notice that $T(n)$ grows as fast as the Fibonacci numbers $F(n)$, so in fact $T(n) = \Theta(F(n))$.

Fibonacci Numbers – Naïve Algorithm

- What's going on with this naïve approach?



- Memoization frees us from redundant calculations 😊
 - Remember solutions of all the sub-problems
 - Trade space for time



Sub-problem	Opt Solution
fib(0)	0
fib(1)	1
fib(2)	1
fib(3)	2
fib(4)	3

- Computing the n^{th} Fibonacci number using as follow:

- $F(0) = 0$
- $F(1) = 1$
- $F(2) = 1+0 = 1$
- ...
- $F(n-2) =$
- $F(n-1) =$
- $F(n) = F(n-1) + F(n-2)$



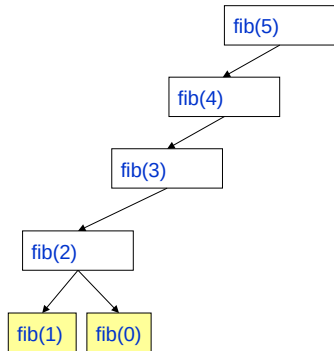
```
def fasterFibonacci(n):  
    F = [0, 1, None, None, ..., None ]  
    \\ F has length n + 1  
    for i = 2, ..., n:  
        F[i] = F[i-1] + F[i-2]  
    return F[n]
```

0	1	1	. . .	$F(n-2)$	$F(n-1)$	$F(n)$
---	---	---	-------	----------	----------	--------

- Efficiency:
 - Time - $O(n)$
 - Space - $O(n)$ → can be improved to $O(1)$

This is an example of **dynamic programming** 😊

- Ideas
 - Ensure all needed recursive calls are already computed and memorized
 - ➔ a good schedule of computation
 - (Optional) Reused space to store previous recursive call results
 - ➔ Arrive at the same efficient (special) solution for Fib()



*“ Those who cannot remember the
past are condemned to repeat it. ”*

— Dynamic Programming

- Dynamic Programming is an algorithm design technique for *optimization problems*: often minimizing or maximizing.
- Like divide and conquer, DP solves problems by **combining solutions to sub-problems**.
- Unlike divide and conquer, sub-problems are **not independent**.
 - DP breaks up a problem into a series of **overlapping** sub-problems.
 - i.e, Both $F[i+1]$ and $F[i+2]$ directly use $F[i]$. And lots of different $F[i+x]$ indirectly use $F[i]$.

1. **Recursion**: Divide the problem into sub-problems, so that their solutions can be combined into a solution to the problem.
2. **Tabulation of sub-problems**: Solve each sub-problem just once and save its solution in a “look-up” table.

- The term Dynamic Programming comes from Control Theory, not computer science. Programming refers to the use of tables (arrays) to construct a solution.
- In Dynamic Programming, we usually reduce time by increasing the amount of **space**.
- We solve the problem by solving sub-problems of increasing size and saving each optimal solution in a table (usually).
- The table is then used for finding the optimal solution to larger problems.
- Time is saved since each sub-problem is solved only once.

Two Ways to Think and Implement DP

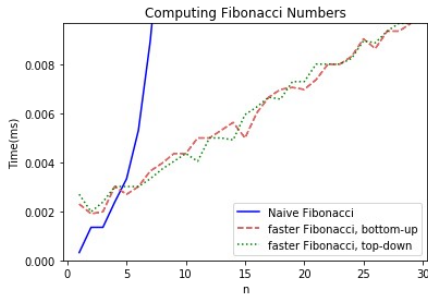
- Top down:
 - Think of it like a recursive algorithm.
 - To solve the big problem:
 - Recurse to solve smaller problems
 - Those recurse to solve smaller problems
 - etc..
 - The difference from divide and conquer:
 - Keep track of what small problems you've already solved to prevent re-solving the same problem twice.
 - Aka, "memoization"
- Bottom up:
 - For Fibonacci:
 - Solve the small problems first
 - fill in $F[0], F[1]$
 - Then bigger problems
 - ...
 - Then bigger problems
 - fill in $F[n-1]$
 - Then finally solve the real problem.
 - fill in $F[n]$

Example of Top-Down Fibonacci

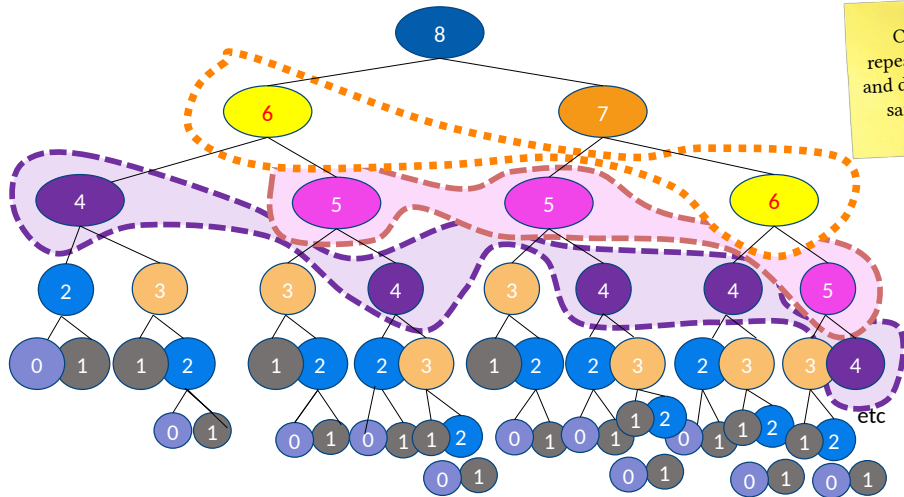
```
define a global list F = [0,1,None, None, ..., None]
```

```
def Fibonacci(n):  
    if F[n] != None:  
        return F[n]  
    else:  
        F[n] = Fibonacci(n-1) + Fibonacci(n-2)  
    return F[n]
```

Memoization: Keeps track (in F) of the stuff you've already done.



Memoization Visualization



Minimum Coin Change

Input

- A set C of positive integer coin values.
- A nonnegative integer target T .
- Each coin value may be used any number of times.

Goal

Find the **minimum number of coins** whose total value is **exactly** T .

Return the minimum number of coins, or -1 if it is impossible.

$$C = \{1, 3, 4\}, \quad T = 6.$$

Question: What is an optimal solution?

A Natural Greedy Attempt

Idea: repeatedly take the largest coin that fits.

$$C = \{1, 3, 4\}, \quad T = 6.$$

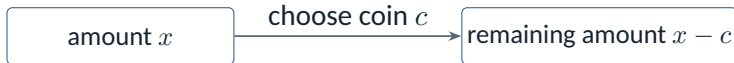
Method	Coins	Number of coins
Largest coin first	$4 + 1 + 1$	3
Optimal solution	$3 + 3$	2

Choosing coin 4 leaves amount 2, which needs two more coins.

We need to compare the total number of coins for each possible choice.

A Choice and Its Remaining Subproblem

To make amount $x > 0$, consider choosing a coin $c \in C$ with $c \leq x$.



For $C = \{1, 3, 4\}$ and $x = 6$:

Chosen coin	Remaining amount	Remaining task
1	5	Make 5 with as few coins as possible
3	3	Make 3 with as few coins as possible
4	2	Make 2 with as few coins as possible

In an optimal solution, suppose it includes a coin of value c ,

- the remaining coins must form an optimal solution for amount $x - c$.

Optimal Substructure Property

Definition (Optimal Substructure)

A problem exhibits optimal substructure if an optimal solution to the problem contains within it optimal solutions to subproblems.

For minimum coin change

- Consider an optimal solution for amount x . Suppose it includes a coin of value c .
- The remaining coins make amount $x - c$. They must form an **optimal solution to the subproblem of making $x - c$** .
- Otherwise, we could make $x - c$ with fewer coins. Adding the coin c would then make x with fewer coins, contradicting optimality.

Subproblems of Minimum Coin Change

For each amount $0 \leq x \leq T$, consider the subproblem:

Find the minimum number of coins whose total value is exactly x .

Its optimal value is

$$D[x] = \text{minimum number of coins with total value } x.$$

- $D[0] = 0$: no coins are needed.
- $D[x] = +\infty$ if amount x cannot be made.

To solve the original problem, we need to compute $D[T]$.

Once computed, return $D[T]$ if it is finite, or -1 if it is infinite.

Recurrence of Minimum Coin Change

$$D[x] = \begin{cases} 0, & x = 0, \\ \min_{\substack{c \in C \\ c \leq x}} (1 + D[x - c]), & 1 \leq x \leq T. \end{cases}$$

- c : a coin value we can choose.
- $1 + D[x - c]$: one coin plus the minimum number needed for the remainder.
- The minimum compares all possible coin choices.

An impossible remainder gives an infinite candidate: $1 + \infty = \infty$.

If no coin fits, $D[x] = +\infty$.

Optimal substructure explains why we use an optimal answer for each remainder.

A Direct Recursive Algorithm

Try each coin, recursively solve the remainder, and keep the best result.

procedure Fewest-Coins(x)

if $x = 0$ **then**

return 0

$best \leftarrow +\infty$

for each $c \in C$ **with** $c \leq x$ **do**

$best \leftarrow \min\{best, 1 + \text{Fewest-Coins}(x - c)\}$

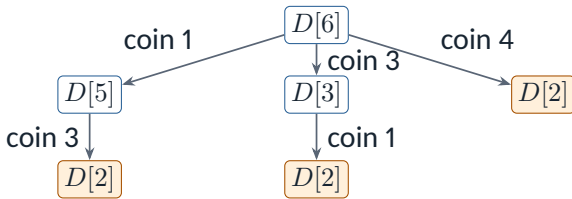
return $best$

Question: This algorithm computes $D[x]$. Is this algorithm efficient?

Overlapping Subproblems Property

The same subproblem occurs in different recursive branches.

Some recursive calls for $C = \{1, 3, 4\}$:



For amount 20: 20,736 recursive calls, but only 21 distinct subproblems.

Each repeated call computes the same answer again, just as in Fibonacci.

Memoization: Saving Recursive Answers

As in Fibonacci, store each answer after computing it.

When a recursive call asks for $D[x]$:

1. If the answer is already stored, return it immediately.
2. Otherwise, compute it recursively and store the answer.

For $C = \{1, 3, 4\}$:

First request for $D[2]$ Compute $D[2] = 2$ and save it.

Later requests for $D[2]$ Return the saved answer without further recursion.

Use the same recurrence. Solve each needed subproblem once.

Top-Down DP with Memoization

Table M stores computed answers: $M[x] = D[x]$.

Initially, $M[0] = 0$ and every other entry is unknown.

procedure Memo-Coins(x)

if $M[x]$ is not unknown **then**

return $M[x]$

$best \leftarrow +\infty$

for each $c \in C$ with $c \leq x$ **do**

$best \leftarrow \min\{best, 1 + \text{Memo-Coins}(x - c)\}$

$M[x] \leftarrow best$

return $M[x]$

Start with Memo-Coins(T). Save each answer and reuse it.

Bottom-Up DP: Smaller Amounts First

We can also arrange the computations in advance.

$$D[x] \text{ uses only } D[x - c], \quad x - c < x.$$

- Start with the known answer $D[0] = 0$.
- Compute $D[1], D[2], \dots, D[T]$ in increasing order.
- At each amount, all answers needed by the recurrence are already stored.

For example, with $C = \{1, 3, 4\}$:

$$D[6] = \min\{1 + D[5], 1 + D[3], 1 + D[2]\}.$$

By the time we reach amount 6, the answers for 5, 3, and 2 are ready.

Computing the Table

For $C = \{1, 3, 4\}$, fill the table from left to right.

x	0	1	2	3	4	5	6
$D[x]$	0	1	2	1	1		

$$D[1] = 1 + D[0] = 1,$$

$$D[2] = 1 + D[1] = 2,$$

$$D[3] = \min\{1 + D[2], 1 + D[0]\} = 1,$$

$$D[4] = \min\{1 + D[3], 1 + D[1], 1 + D[0]\} = 1.$$

Question: What are $D[5]$ and $D[6]$?

Choosing the Best Candidate

For $C = \{1, 3, 4\}$:

$$\begin{aligned} D[5] &= \min\{1 + D[4], 1 + D[2], 1 + D[1]\} \\ &= \min\{2, 3, 2\} = 2, \end{aligned}$$

$$\begin{aligned} D[6] &= \min\{1 + D[5], 1 + D[3], 1 + D[2]\} \\ &= \min\{3, 2, 3\} = \mathbf{2}. \end{aligned}$$

x	0	1	2	3	4	5	6
$D[x]$	0	1	2	1	1	2	2

For amount 6, choose coin 3. The remaining amount 3 needs one more coin.

Practice: Coins 2 and 5

Let $C = \{2, 5\}$.

x	0	1	2	3	4	5	6	7
$D[x]$	0							

1. Fill the table, including unreachable amounts.
2. Which coin choices attain $D[7]$?
3. Why must the remaining coins form an optimal solution for the remainder?

Question: Why can we compute these entries from left to right?

Practice Solution: Coins 2 and 5

x	0	1	2	3	4	5	6	7
$D[x]$	0	∞	1	∞	2	1	3	2

$$D[7] = \min\{1 + D[5], 1 + D[2]\} = 2.$$

Both choices produce the same collection of coins: one 2 and one 5.

- Removing coin 2 leaves the optimal one-coin solution for amount 5.
- Removing coin 5 leaves the optimal one-coin solution for amount 2.

Exercise: Bottom-Up Coin Change

Give pseudocode for a bottom-up algorithm using the recurrence for $D[x]$.

1. Start with $D[0] = 0$. Initialize each other entry to $+\infty$.
2. Fill $D[1], \dots, D[T]$, trying every eligible coin at each amount.
3. Return the minimum number of coins, or -1 .

Test	C	T	Expected result
Zero target	$\{1, 3, 4\}$	0	0
Reachable amount	$\{1, 3, 4\}$	6	2
Unreachable amount	$\{4, 6\}$	7	-1

Bottom-Up Coin Change

```
procedure Min-Coins( $C, T$ )  
     $D[0] \leftarrow 0$   
    for  $x \leftarrow 1$  to  $T$  do  
         $D[x] \leftarrow +\infty$   
        for each  $c \in C$  with  $c \leq x$  do  
             $D[x] \leftarrow \min\{D[x], 1 + D[x - c]\}$   
    if  $D[T] = +\infty$  then  
        return  $-1$   
    else  
        return  $D[T]$ 
```

Positive coin values give $x - c < x$. Every needed answer is already stored.

Top-Down and Bottom-Up DP

	Top-down	Bottom-up
Start	Target amount T	Base case $D[0] = 0$
Order	Determined by recursion	Increasing amounts
Saved answers	Table M	Table D
Subproblems	Only requested amounts	All amounts $0, \dots, T$

There are $T + 1$ possible subproblems. Compute each needed answer once, trying at most $m = |C|$ coin values per subproblem.

The total running time of both algorithm is $O(m \cdot T)$, and the space is $O(T)$.

The Two Elements of Dynamic Programming

Optimal substructure

Removing coin c from an optimal solution for x leaves an optimal solution for $x - c$.

This justifies using the best subproblem answers in the recurrence.

Overlapping subproblems

Different recursive branches ask for the same amount.

Saving an answer avoids solving that subproblem again.

For coin change, DP combines optimal subproblem answers and reuses them.

Developing a DP Algorithm [CLRS]

1. Characterize the structure of an optimal solution.

Removing coin c from an optimal solution for x leaves an optimal solution for $x - c$.

2. Recursively define the optimal value.

Define $D[x]$, then specify its recurrence and base cases.

3. Compute the optimal value.

Reuse stored results, using memoization or a bottom-up order.

4. Construct an optimal solution.

Use the saved choices to recover an optimal collection of coins.

Question: The last step is optional, but how?

Remembering the Best Coin Choice

When computing $D[x]$:

- Compare $1 + D[x - c]$ for every coin value $c \in C$ with $c \leq x$.
- Store the coin value c^* that gives the smallest finite candidate as $\text{choice}[x]$.

For $C = \{1, 3, 4\}$ and $x = 6$:

Coin value c	1	3	4
Candidate $1 + D[6 - c]$	3	2	3

$$D[6] = 2, \quad \text{choice}[6] = 3.$$

Record the best choice in both top-down and bottom-up DP.

Recovering the Coins: Pseudocode

After computing $D[T]$, follow the saved choices until the remaining amount is zero.

procedure Recover-Coins(T, D, choice)

if $D[T] = +\infty$ **then**

return -1

$\text{coins} \leftarrow$ empty list

$x \leftarrow T$

while $x > 0$ **do**

$c \leftarrow \text{choice}[x]$

 Append c to coins

$x \leftarrow x - c$

return coins

For $T = 6$, $\text{choice}[6] = 3$ and $\text{choice}[3] = 3$: the output is $[3, 3]$.